

Section 84 Editorial — Master Dynamic Structures Mixed Contest

A. Quantile Prefix Sum

How to find it

Alongside zero counts, store prefix sums of the zero subsequence at every wavelet level.

Correctness

Whenever k passes the zero block, all those values are among the k smallest and their sum is added once. The remaining path ends at one equal value class supplying the final copies.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Wavelet {
    int B, n;
    vector<ll> vals;
    vector<vector<int>> z;
    vector<vector<ll>> zs;
    vector<int> mid;
    Wavelet(vector<ll> a) : n(a.size()) {
        vals = a;
        sort(vals.begin(), vals.end());
        vals.erase(unique(vals.begin(), vals.end()), vals.end());
        B = max(1, (int)bit_width((unsigned)max(1, (int)vals.size() - 1)));
        vector<int> r(n);
        for (int i = 0; i < n; i++)
            r[i] = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
        for (int b = B - 1; b >= 0; b--) {
            z.push_back(vector<int>(n + 1));
            zs.push_back(vector<ll>(n + 1));
            vector<int> r0, r1;
            vector<ll> a0, a1;
            for (int i = 0; i < n; i++) {
                bool zero = !(r[i] >> b & 1);
                z.back()[i + 1] = z.back()[i] + zero;
                zs.back()[i + 1] = zs.back()[i] + (zero ? a[i] : 0);
                (zero ? r0 : r1).push_back(r[i]);
                (zero ? a0 : a1).push_back(a[i]);
            }
            mid.push_back(r0.size());
            r0.insert(r0.end(), r1.begin(), r1.end());
            a0.insert(a0.end(), a1.begin(), a1.end());
        }
    }
};
```

```

        r.swap(r0);
        a.swap(a0);
    }
}

ll kth(int l, int r, int k) {
    int rank = 0;
    for (int d = 0; d < B; d++) {
        int q = z[d][r] - z[d][l], b = B - 1 - d;
        if (k <= q)
            l = z[d][l], r = z[d][r];
        else
            k -= q, rank |= 1 << b, l = mid[d] + l - z[d][l],
            r = mid[d] + r - z[d][r];
    }
    return vals[rank];
}

int lte(int l, int r, ll x) {
    int lim = upper_bound(vals.begin(), vals.end(), x) - vals.begin();
    if (lim == (int)vals.size())
        return r - 1;
    int ans = 0;
    for (int d = 0; d < B; d++) {
        int b = B - 1 - d;
        if (lim >> b & 1)
            ans += z[d][r] - z[d][l], l = mid[d] + l - z[d][l],
            r = mid[d] + r - z[d][r];
        else
            l = z[d][l], r = z[d][r];
    }
    return ans;
}

ll sumk(int l, int r, int k) {
    ll ans = 0;
    int rank = 0;
    for (int d = 0; d < B; d++) {
        int q = z[d][r] - z[d][l], b = B - 1 - d;
        if (k <= q)
            l = z[d][l], r = z[d][r];
        else
            ans += zs[d][r] - zs[d][l], k -= q, rank |= 1 << b,
            l = mid[d] + l - z[d][l], r = mid[d] + r - z[d][r];
    }
    return ans + vals[rank] * k;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    Wavelet w(a);
    while (q--) {
        int l, r, k;
        cin >> l >> r >> k;
        cout << w.sumk(l - 1, r, k) << '\n';
    }
}

```

Python

```
import sys
from array import array

class Wavelet:
    def __init__(self, a):
        self.values = sorted(set(a))
        rank = {x: i for i, x in enumerate(self.values)}
        r = [rank[x] for x in a]
        self.bits = max(1, (len(self.values) - 1).bit_length())
        self.zero = []
        self.zero_sum = []
        self.mid = []
        v = a[:]
        for bit in range(self.bits - 1, -1, -1):
            pc = array("I", [0])
            ps = array("q", [0])
            zr = []
            zv = []
            orank = []
            oval = []
            for x, y in zip(r, v):
                z = not (x >> bit & 1)
                pc.append(pc[-1] + z)
                ps.append(ps[-1] + (y if z else 0))
                (zr if z else orank).append(x)
                (zv if z else oval).append(y)
            self.zero.append(pc)
            self.zero_sum.append(ps)
            self.mid.append(len(zr))
            r = zr + orank
            v = zv + oval

    def kth(self, l, r, k):
        rank = 0
        for level in range(self.bits):
            bit = self.bits - 1 - level
            p = self.zero[level]
            zl = p[r] - p[l]
            if k <= zl:
                l, r = p[l], p[r]
            else:
                k -= zl
                rank |= 1 << bit
                l, r = self.mid[level] + l - p[l], self.mid[level] + r - p[r]
        return self.values[rank]

    def lte(self, l, r, x):
        import bisect

        limit = bisect.bisect_right(self.values, x)
        if limit >= len(self.values):
            return r - l
        ans = 0
        for level in range(self.bits):
            bit = self.bits - 1 - level
            p = self.zero[level]
            if limit >> bit & 1:
                ans += p[r] - p[l]
                l, r = self.mid[level] + l - p[l], self.mid[level] + r - p[r]
            else:
```

```

        l, r = p[l], p[r]
    return ans

def sum_k(self, l, r, k):
    ans = 0
    rank = 0
    for level in range(self.bits):
        bit = self.bits - 1 - level
        p = self.zero[level]
        s = self.zero_sum[level]
        zl = p[r] - p[l]
        if k <= zl:
            l, r = p[l], p[r]
        else:
            ans += s[r] - s[l]
            k -= zl
            rank |= 1 << bit
            l, r = self.mid[level] + l - p[l], self.mid[level] + r - p[r]
    return ans + self.values[rank] * k

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, q = d[:2]
    a = d[2 : 2 + n]
    w = Wavelet(a)
    at = 2 + n
    print(
        "\n".join(
            str(w.sum_k(d[i] - 1, d[i + 1], d[i + 2]))
            for i in range(at, at + 3 * q, 3)
        )
    )

if __name__ == "__main__":
    main()

```

B. Clamped Terrain

How to find it

Maintain symmetric maximum and minimum beats metadata and push both inherited bounds.

Correctness

Each fast upper or lower operation changes one extremal value class exactly. Recursive cases partition until every changed class is handled, while push preserves all ancestor clamps.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 4e18;
struct Beats {
    struct N {
        ll sum = 0, mx = -INF, smx = -INF, mn = INF, smn = INF;
        int mxc = 0, mnc = 0;
    };
    int n;
    vector<N> t;
    Beats(vector<ll> a) : n(a.size()), t(4 * n) { build(1, 0, n, a); }
    void build(int p, int l, int r, vector<ll> &a) {
        if (r - l == 1) {
            t[p] = {a[l], a[l], -INF, a[l], INF, 1, 1};
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
    void pull(int p) {
        N &a = t[p * 2], &b = t[p * 2 + 1], &x = t[p];
        x.sum = a.sum + b.sum;
        if (a.mx > b.mx)
            x.mx = a.mx, x.mxc = a.mxc, x.smx = max(a.smx, b.mx);
        else if (a.mx < b.mx)
            x.mx = b.mx, x.mxc = b.mxc, x.smx = max(a.mx, b.smx);
        else
            x.mx = a.mx, x.mxc = a.mxc + b.mxc, x.smx = max(a.smx, b.smx);
        if (a.mn < b.mn)
            x.mn = a.mn, x.mnc = a.mnc, x.smn = min(a.smn, b.mn);
        else if (a.mn > b.mn)
            x.mn = b.mn, x.mnc = b.mnc, x.smn = min(a.mn, b.smn);
        else
            x.mn = a.mn, x.mnc = a.mnc + b.mnc, x.smn = min(a.smn, b.smn);
    }
    void upper_node(int p, ll x) {
        if (t[p].mx <= x)
            return;
        t[p].sum += (x - t[p].mx) * t[p].mxc;
        if (t[p].mn == t[p].mx)
            t[p].mn = x;
        else if (t[p].smn == t[p].mx)
            t[p].smn = x;
        t[p].mx = x;
    }
    void lower_node(int p, ll x) {
        if (t[p].mn >= x)
            return;
        t[p].sum += (x - t[p].mn) * t[p].mnc;
        if (t[p].mx == t[p].mn)
            t[p].mx = x;
        else if (t[p].smx == t[p].mn)
            t[p].smx = x;
        t[p].mn = x;
    }
    void push(int p) {
        upper_node(p * 2, t[p].mx);
        upper_node(p * 2 + 1, t[p].mx);
        lower_node(p * 2, t[p].mn);
        lower_node(p * 2 + 1, t[p].mn);
    }
};

```

```

}
void upper(int ql, int qr, ll x, int p = 1, int l = 0, int r = -1) {
    if (r < 0)
        r = n;
    if (qr <= l || r <= ql || t[p].mx <= x)
        return;
    if (ql <= l && r <= qr && t[p].smx < x) {
        upper_node(p, x);
        return;
    }
    push(p);
    int m = (l + r) / 2;
    upper(ql, qr, x, p * 2, l, m);
    upper(ql, qr, x, p * 2 + 1, m, r);
    pull(p);
}

void lower(int ql, int qr, ll x, int p = 1, int l = 0, int r = -1) {
    if (r < 0)
        r = n;
    if (qr <= l || r <= ql || t[p].mn >= x)
        return;
    if (ql <= l && r <= qr && t[p].smn > x) {
        lower_node(p, x);
        return;
    }
    push(p);
    int m = (l + r) / 2;
    lower(ql, qr, x, p * 2, l, m);
    lower(ql, qr, x, p * 2 + 1, m, r);
    pull(p);
}

ll query(int ql, int qr, int p = 1, int l = 0, int r = -1) {
    if (r < 0)
        r = n;
    if (qr <= l || r <= ql)
        return 0;
    if (ql <= l && r <= qr)
        return t[p].sum;
    push(p);
    int m = (l + r) / 2;
    return query(ql, qr, p * 2, l, m) + query(ql, qr, p * 2 + 1, m, r);
}

};

struct ModTree {
    int n;
    vector<ll> s, mx;
    ModTree(vector<ll> a) : n(a.size()), s(4 * n), mx(4 * n) {
        build(1, 0, n, a);
    }
    void build(int p, int l, int r, vector<ll> &a) {
        if (r - l == 1) {
            s[p] = mx[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
    void pull(int p) {
        s[p] = s[p * 2] + s[p * 2 + 1];
        mx[p] = max(mx[p * 2], mx[p * 2 + 1]);
    }
}

```

```

void mod(int ql, int qr, ll x, int p = 1, int l = 0, int r = -1) {
    if (r < 0)
        r = n;
    if (qr <= 1 || r <= ql || mx[p] < x)
        return;
    if (r - l == 1) {
        s[p] %= x;
        mx[p] = s[p];
        return;
    }
    int m = (l + r) / 2;
    mod(ql, qr, x, p * 2, l, m);
    mod(ql, qr, x, p * 2 + 1, m, r);
    pull(p);
}

void setv(int i, ll x, int p = 1, int l = 0, int r = -1) {
    if (r < 0)
        r = n;
    if (r - l == 1) {
        s[p] = mx[p] = x;
        return;
    }
    int m = (l + r) / 2;
    i < m ? setv(i, x, p * 2, l, m) : setv(i, x, p * 2 + 1, m, r);
    pull(p);
}

ll query(int ql, int qr, int p = 1, int l = 0, int r = -1) {
    if (r < 0)
        r = n;
    if (qr <= 1 || r <= ql)
        return 0;
    if (ql <= 1 && r <= qr)
        return s[p];
    int m = (l + r) / 2;
    return query(ql, qr, p * 2, l, m) + query(ql, qr, p * 2 + 1, m, r);
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    Beats t(a);
    while (q--) {
        string op;
        int l, r;
        cin >> op >> l >> r;
        if (op == "UPPER") {
            ll x;
            cin >> x;
            t.upper(l - 1, r, x);
        } else if (op == "LOWER") {
            ll x;
            cin >> x;
            t.lower(l - 1, r, x);
        } else
            cout << t.query(l - 1, r) << '\n';
    }
}

```

Python

```
import sys

INF = 10**30

class Beats:
    def __init__(self, a):
        n = len(a)
        z = 4 * n
        self.n = n
        self.sum = [0] * z
        self.mx = [-INF] * z
        self.smx = [-INF] * z
        self.mxc = [0] * z
        self.mn = [INF] * z
        self.smn = [INF] * z
        self.mnc = [0] * z
        self._build(1, 0, n, a)

    def _build(self, p, l, r, a):
        if r - l == 1:
            self.sum[p] = self.mx[p] = self.mn[p] = a[l]
            self.mxc[p] = self.mnc[p] = 1
            return
        m = (l + r) // 2
        self._build(p * 2, l, m, a)
        self._build(p * 2 + 1, m, r, a)
        self._pull(p)

    def _pull(self, p):
        a, b = p * 2, p * 2 + 1
        self.sum[p] = self.sum[a] + self.sum[b]
        if self.mx[a] > self.mx[b]:
            self.mx[p], self.mxc[p], self.smx[p] = (
                self.mx[a],
                self.mxc[a],
                max(self.smx[a], self.mx[b]),
            )
        elif self.mx[a] < self.mx[b]:
            self.mx[p], self.mxc[p], self.smx[p] = (
                self.mx[b],
                self.mxc[b],
                max(self.mx[a], self.smx[b]),
            )
        else:
            self.mx[p], self.mxc[p], self.smx[p] = (
                self.mx[a],
                self.mxc[a] + self.mxc[b],
                max(self.smx[a], self.smx[b]),
            )
        if self.mn[a] < self.mn[b]:
            self.mn[p], self.mnc[p], self.smn[p] = (
                self.mn[a],
                self.mnc[a],
                min(self.smn[a], self.mn[b]),
            )
        elif self.mn[a] > self.mn[b]:
            self.mn[p], self.mnc[p], self.smn[p] = (
                self.mn[b],
                self.mnc[b],
                min(self.mn[a], self.smn[b]),
            )
```



```

    )
    else:
        self.mn[p], self.mnc[p], self.smn[p] = (
            self.mn[a],
            self.mnc[a] + self.mnc[b],
            min(self.smn[a], self.smn[b]),
        )

def _upper_node(self, p, x):
    if self.mx[p] <= x:
        return
    self.sum[p] += (x - self.mx[p]) * self.mxc[p]
    if self.mn[p] == self.mx[p]:
        self.mn[p] = x
    elif self.smn[p] == self.mx[p]:
        self.smn[p] = x
    self.mx[p] = x

def _lower_node(self, p, x):
    if self.mn[p] >= x:
        return
    self.sum[p] += (x - self.mn[p]) * self.mnc[p]
    if self.mx[p] == self.mn[p]:
        self.mx[p] = x
    elif self.smx[p] == self.mn[p]:
        self.smx[p] = x
    self.mn[p] = x

def _push(self, p):
    for c in (p * 2, p * 2 + 1):
        self._upper_node(c, self.mx[p])
        self._lower_node(c, self.mn[p])

def upper(self, ql, qr, x, p=1, l=0, r=None):
    if r is None:
        r = self.n
    if qr <= l or r <= ql or self.mx[p] <= x:
        return
    if ql <= l and r <= qr and self.smx[p] < x:
        self._upper_node(p, x)
        return
    self._push(p)
    m = (l + r) // 2
    self.upper(ql, qr, x, p * 2, l, m)
    self.upper(ql, qr, x, p * 2 + 1, m, r)
    self._pull(p)

def lower(self, ql, qr, x, p=1, l=0, r=None):
    if r is None:
        r = self.n
    if qr <= l or r <= ql or self.mn[p] >= x:
        return
    if ql <= l and r <= qr and self.smn[p] > x:
        self._lower_node(p, x)
        return
    self._push(p)
    m = (l + r) // 2
    self.lower(ql, qr, x, p * 2, l, m)
    self.lower(ql, qr, x, p * 2 + 1, m, r)
    self._pull(p)

def query(self, ql, qr, p=1, l=0, r=None):
    if r is None:

```

```

        r = self.n
        if qr <= 1 or r <= ql:
            return 0
        if ql <= 1 and r <= qr:
            return self.sum[p]
        self._push(p)
        m = (1 + r) // 2
        return self.query(ql, qr, p * 2, l, m) + self.query(
            ql, qr, p * 2 + 1, m, r
        )

```

```
class ModTree:
```

```

    def __init__(self, a):
        self.n = len(a)
        self.s = [0] * (4 * self.n)
        self.mx = [0] * (4 * self.n)
        self._build(1, 0, self.n, a)

    def _build(self, p, l, r, a):
        if r - l == 1:
            self.s[p] = self.mx[p] = a[l]
            return
        m = (1 + r) // 2
        self._build(p * 2, l, m, a)
        self._build(p * 2 + 1, m, r, a)
        self._pull(p)

    def _pull(self, p):
        self.s[p] = self.s[p * 2] + self.s[p * 2 + 1]
        self.mx[p] = max(self.mx[p * 2], self.mx[p * 2 + 1])

    def mod(self, ql, qr, x, p=1, l=0, r=None):
        if r is None:
            r = self.n
        if qr <= l or r <= ql or self.mx[p] < x:
            return
        if r - l == 1:
            self.s[p] %= x
            self.mx[p] = self.s[p]
            return
        m = (1 + r) // 2
        self.mod(ql, qr, x, p * 2, l, m)
        self.mod(ql, qr, x, p * 2 + 1, m, r)
        self._pull(p)

    def set(self, i, x, p=1, l=0, r=None):
        if r is None:
            r = self.n
        if r - l == 1:
            self.s[p] = self.mx[p] = x
            return
        m = (1 + r) // 2
        if i < m:
            self.set(i, x, p * 2, l, m)
        else:
            self.set(i, x, p * 2 + 1, m, r)
        self._pull(p)

    def query(self, ql, qr, p=1, l=0, r=None):
        if r is None:
            r = self.n
        if qr <= l or r <= ql:

```

```

        return 0
    if ql <= l and r <= qr:
        return self.s[p]
    m = (l + r) // 2
    return self.query(ql, qr, p * 2, l, m) + self.query(
        ql, qr, p * 2 + 1, m, r
    )

def main():
    d = sys.stdin.buffer.read().split()
    n, q = map(int, d[:2])
    t = Beats(list(map(int, d[2 : 2 + n])))
    at = 2 + n
    out = []
    for _ in range(q):
        op = d[at]
        l = int(d[at + 1]) - 1
        r = int(d[at + 2])
        at += 3
        if op == b"UPPER":
            t.upper(l, r, int(d[at]))
            at += 1
        elif op == b"LOWER":
            t.lower(l, r, int(d[at]))
            at += 1
        else:
            out.append(str(t.query(l, r)))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

C. Reversible String Hash

How to find it

An implicit treap stores forward and reverse polynomial hashes; reversal swaps both children and hashes.

Correctness

The in-order hash recurrence is exact after every pull. Lazy reversal produces the reverse sequence and swaps its two hashes, so equality is exactly the maintained palindrome certificate.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1000000007, BASE = 911382323;
vector<ll> pw = {1};
mt19937 rng(712367821);
struct N {

```

```

    ll v, sum, hf, hr, add = 0;
    uint32_t p = rng();
    int z = 1;
    bool rev = 0;
    N *l = 0, *r = 0;
    N(ll v) : v(v), sum(v), hf((v % MOD + MOD) % MOD), hr(hf) {}
};

int sz(N *t) { return t ? t->z : 0; }
ll sm(N *t) { return t ? t->sum : 0; }
void ensure(int n) {
    while ((int)pw.size() <= n)
        pw.push_back(pw.back() * BASE % MOD);
}
void pull(N *t) {
    if (!t)
        return;
    ensure(sz(t->l) + sz(t->r) + 1);
    t->z = 1 + sz(t->l) + sz(t->r);
    t->sum = t->v + sm(t->l) + sm(t->r);
    int l = sz(t->l), r = sz(t->r);
    ll lf = t->l ? t->l->hf : 0, rf = t->r ? t->r->hf : 0,
        lr = t->l ? t->l->hr : 0, rr = t->r ? t->r->hr : 0;
    t->hf = (lf + t->v * pw[l] + rf * pw[l + 1]) % MOD;
    t->hr = (rr + t->v * pw[r] + lr * pw[r + 1]) % MOD;
}
void add(N *t, ll x) {
    if (t)
        t->v += x, t->sum += x * t->z, t->add += x;
}
void reverse_node(N *t) {
    if (t)
        swap(t->l, t->r), swap(t->hf, t->hr), t->rev ^= 1;
}
void push(N *t) {
    if (!t)
        return;
    if (t->rev)
        reverse_node(t->l), reverse_node(t->r), t->rev = 0;
    if (t->add)
        add(t->l, t->add), add(t->r, t->add), t->add = 0;
}
pair<N *, N *> split(N *t, int k) {
    if (!t)
        return {};
    push(t);
    if (sz(t->l) >= k) {
        auto [a, b] = split(t->l, k);
        t->l = b;
        pull(t);
        return {a, t};
    }
    auto [a, b] = split(t->r, k - sz(t->l) - 1);
    t->r = a;
    pull(t);
    return {t, b};
}
N *merge(N *a, N *b) {
    if (!a || !b)
        return a ? a : b;
    if (a->p > b->p) {
        push(a);
        a->r = merge(a->r, b);
        pull(a);
    }

```

```

    return a;
}
push(b);
b->l = merge(a, b->l);
pull(b);
return b;
}
void output(N *t) {
    if (!t)
        return;
    push(t);
    output(t->l);
    cout << t->v << ' ';
    output(t->r);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    string s;
    cin >> n >> q >> s;
    N *root = 0;
    for (char c : s)
        root = merge(root, new N(c - 'a' + 1));
    while (q--) {
        string op;
        cin >> op;
        if (op == "SET") {
            int i;
            char c;
            cin >> i >> c;
            auto [a, x] = split(root, i - 1);
            auto [b, d] = split(x, 1);
            b->v = c - 'a' + 1;
            pull(b);
            root = merge(merge(a, b), d);
        } else {
            int l, r;
            cin >> l >> r;
            auto [a, x] = split(root, l - 1);
            auto [b, c] = split(x, r - l + 1);
            if (op == "REV")
                reverse_node(b);
            else
                cout << (b->hf == b->hr ? "YES\n" : "NO\n");
            root = merge(merge(a, b), c);
        }
    }
}

```

Python

```

import sys

sys.setrecursionlimit(1_000_000)
MOD = 1_000_000_007
BASE = 911382323
POW = [1]
seed = 712367821

def rnd():

```

```

global seed
seed ^= seed << 13 & 0xFFFFFFFF
seed ^= seed >> 17
seed ^= seed << 5 & 0xFFFFFFFF
return seed & 0xFFFFFFFF

class Node:
    __slots__ = "v", "p", "l", "r", "z", "sum", "add", "rev", "hf", "hr"

    def __init__(self, v):
        self.v = v
        self.p = rnd()
        self.l = self.r = None
        self.z = 1
        self.sum = v
        self.add = 0
        self.rev = False
        self.hf = self.hr = v % MOD

def size(t):
    return t.z if t else 0

def total(t):
    return t.sum if t else 0

def ensure(n):
    while len(POW) <= n:
        POW.append(POW[-1] * BASE % MOD)

def pull(t):
    if not t:
        return
    ensure(size(t.l) + size(t.r) + 1)
    t.z = 1 + size(t.l) + size(t.r)
    t.sum = t.v + total(t.l) + total(t.r)
    ll = size(t.l)
    rr = size(t.r)
    lf = t.l.hf if t.l else 0
    rf = t.r.hf if t.r else 0
    lr = t.l.hr if t.l else 0
    rrrh = t.r.hr if t.r else 0
    t.hf = (lf + t.v * POW[ll] + rf * POW[ll + 1]) % MOD
    t.hr = (rrh + t.v * POW[rr] + lr * POW[rr + 1]) % MOD

def apply_add(t, x):
    if t:
        t.v += x
        t.sum += x * t.z
        t.add += x

def apply_rev(t):
    if t:
        t.l, t.r = t.r, t.l
        t.hf, t.hr = t.hr, t.hf
        t.rev ^= True

```

```

def push(t):
    if not t:
        return
    if t.rev:
        apply_rev(t.l)
        apply_rev(t.r)
        t.rev = False
    if t.add:
        apply_add(t.l, t.add)
        apply_add(t.r, t.add)
        t.add = 0

def split(t, k):
    if not t:
        return None, None
    push(t)
    if size(t.l) >= k:
        a, t.l = split(t.l, k)
        pull(t)
        return a, t
    t.r, b = split(t.r, k - size(t.l) - 1)
    pull(t)
    return t, b

def merge(a, b):
    if not a or not b:
        return a or b
    if a.p > b.p:
        push(a)
        a.r = merge(a.r, b)
        pull(a)
        return a
    push(b)
    b.l = merge(a, b.l)
    pull(b)
    return b

def build(values):
    root = None
    for x in values:
        root = merge(root, Node(x))
    return root

def values(t, out):
    if not t:
        return
    push(t)
    values(t.l, out)
    out.append(t.v)
    values(t.r, out)

def main():
    d = sys.stdin.buffer.read().split()
    n, q = map(int, d[:2])
    root = build(ord(c) - 96 for c in d[2].decode())
    at = 3
    out = []
    for _ in range(q):

```

```

op = d[at]
at += 1
if op == b"SET":
    i = int(d[at]) - 1
    x = ord(d[at + 1]) - 96
    at += 2
    a, b = split(root, i)
    b, c = split(b, 1)
    b.v = x
    pull(b)
    root = merge(merge(a, b), c)
else:
    l = int(d[at]) - 1
    r = int(d[at + 1])
    at += 2
    a, b = split(root, l)
    b, c = split(b, r - l)
    if op == b"REV":
        apply_rev(b)
    else:
        out.append("YES" if b.hf == b.hr else "NO")
    root = merge(merge(a, b), c)
print("\n".join(out))

if __name__ == "__main__":
    main()

```

D. Dynamic Forest Sum

How to find it

Use path exposure exactly as for XOR, but store additive aggregates and point assignments.

Correctness

Exposure selects exactly the represented path and splay pulls maintain its vertex sum, hence every query returns precisely those vertices once.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct LCT {
    int n, mode;
    ll mod;
    vector<array<int, 2>> ch;
    vector<int> p, rev, sz;
    vector<ll> val, agg, mul, add;
    LCT(vector<ll> a, int mode = 0, ll mod = 0)
        : n(a.size()), mode(mode), mod(mod), ch(n + 1), p(n + 1), rev(n + 1),
          sz(n + 1, 1), val(n + 1), agg(n + 1), mul(n + 1, 1), add(n + 1) {}
    sz[0] = 0;

```



```

    for (int i = 1; i <= n; i++)
        val[i] = agg[i] = a[i - 1];
}
bool root(int x) { return !p[x] || (ch[p[x]][0] != x && ch[p[x]][1] != x); }
void pull(int x) {
    auto [l, r] = ch[x];
    sz[x] = 1 + sz[l] + sz[r];
    agg[x] = mode ? (agg[l] ^ val[x] ^ agg[r]) : agg[l] + val[x] + agg[r];
    if (mod)
        agg[x] %= mod;
}
void reverse_node(int x) {
    if (x)
        swap(ch[x][0], ch[x][1]), rev[x] ^= 1;
}
void affine(int x, ll a, ll b) {
    if (!x)
        return;
    val[x] = (a * val[x] + b) % mod;
    agg[x] = (a * agg[x] + b * sz[x]) % mod;
    mul[x] = a * mul[x] % mod;
    add[x] = (a * add[x] + b) % mod;
}
void push(int x) {
    if (rev[x])
        reverse_node(ch[x][0]), reverse_node(ch[x][1]), rev[x] = 0;
    if (mul[x] != 1 || add[x]) {
        affine(ch[x][0], mul[x], add[x]);
        affine(ch[x][1], mul[x], add[x]);
        mul[x] = 1;
        add[x] = 0;
    }
}
void rotate(int x) {
    int y = p[x], z = p[y], s = ch[y][1] == x, b = ch[x][s ^ 1];
    if (!root(y))
        ch[z][ch[z][1] == y] = x;
    p[x] = z;
    ch[x][s ^ 1] = y;
    p[y] = x;
    ch[y][s] = b;
    if (b)
        p[b] = y;
    pull(y);
    pull(x);
}
void splay(int x) {
    vector<int> st = {x};
    for (int y = x; !root(y);)
        y = p[y], st.push_back(y);
    while (!st.empty())
        push(st.back()), st.pop_back();
    while (!root(x)) {
        int y = p[x], z = p[y];
        if (!root(y))
            rotate((ch[y][1] == x) == (ch[z][1] == y) ? y : x);
        rotate(x);
    }
}
void access(int x) {
    for (int y = 0, z = x; z; y = z, z = p[z])
        splay(z), ch[z][1] = y, pull(z);
    splay(x);
}

```

```

}
void makeroot(int x) {
    access(x);
    reverse_node(x);
}
void link(int x, int y) {
    makeroot(x);
    p[x] = y;
}
void cut(int x, int y) {
    makeroot(x);
    access(y);
    ch[y][0] = 0;
    p[x] = 0;
    pull(y);
}
int path(int x, int y) {
    makeroot(x);
    access(y);
    return y;
}
void setv(int x, ll v) {
    access(x);
    val[x] = v;
    pull(x);
}
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    LCT t(a, 0, 0);
    while (q--) {
        string op;
        cin >> op;
        if (op == "LINK") {
            int u, v;
            cin >> u >> v;
            t.link(u, v);
        } else if (op == "CUT") {
            int u, v;
            cin >> u >> v;
            t.cut(u, v);
        } else if (op == "SET") {
            int u;
            ll x;
            cin >> u >> x;
            t.setv(u, x);
        } else if (op == "AFFINE") {
            int u, v;
            ll x, y;
            cin >> u >> v >> x >> y;
            int z = t.path(u, v);
            t.affine(z, x, y);
        } else {
            int u, v;
            cin >> u >> v;
            cout << t.agg[t.path(u, v)] << '\n';
        }
    }
}

```

```
}  
}
```

Python

```
import sys  
  
class LinkCut:  
    def __init__(self, values, mode="sum", mod=None):  
        n = len(values)  
        self.n = n  
        self.ch = [[0, 0] for _ in range(n + 1)]  
        self.p = [0] * (n + 1)  
        self.rev = [0] * (n + 1)  
        self.val = [0] + values[:]  
        self.agg = [0] + values[:]  
        self.sz = [0] + [1] * n  
        self.mode = mode  
        self.mod = mod  
        self.mul = [1] * (n + 1)  
        self.add = [0] * (n + 1)  
  
    def root(self, x):  
        p = self.p[x]  
        return not p or (self.ch[p][0] != x and self.ch[p][1] != x)  
  
    def pull(self, x):  
        l, r = self.ch[x]  
        self.sz[x] = 1 + self.sz[l] + self.sz[r]  
        self.agg[x] = (  
            (self.agg[l] ^ self.val[x] ^ self.agg[r])  
            if self.mode == "xor"  
            else self.agg[l] + self.val[x] + self.agg[r]  
        )  
        if self.mod:  
            self.agg[x] %= self.mod  
  
    def reverse(self, x):  
        if x:  
            self.ch[x].reverse()  
            self.rev[x] ^= 1  
  
    def affine(self, x, a, b):  
        if not x:  
            return  
        mod = self.mod  
        self.val[x] = (a * self.val[x] + b) % mod  
        self.agg[x] = (a * self.agg[x] + b * self.sz[x]) % mod  
        self.mul[x] = a * self.mul[x] % mod  
        self.add[x] = (a * self.add[x] + b) % mod  
  
    def push(self, x):  
        if self.rev[x]:  
            self.reverse(self.ch[x][0])  
            self.reverse(self.ch[x][1])  
            self.rev[x] = 0  
        if self.mul[x] != 1 or self.add[x]:  
            a, b = self.mul[x], self.add[x]  
            self.affine(self.ch[x][0], a, b)  
            self.affine(self.ch[x][1], a, b)
```

```

        self.mul[x] = 1
        self.add[x] = 0

def rotate(self, x):
    p = self.p[x]
    g = self.p[p]
    side = self.ch[p][1] == x
    b = self.ch[x][side ^ 1]
    if not self.root(p):
        self.ch[g][self.ch[g][1] == p] = x
    self.p[x] = g
    self.ch[x][side ^ 1] = p
    self.p[p] = x
    self.ch[p][side] = b
    if b:
        self.p[b] = p
    self.pull(p)
    self.pull(x)

def splay(self, x):
    path = [x]
    y = x
    while not self.root(y):
        y = self.p[y]
        path.append(y)
    for y in reversed(path):
        self.push(y)
    while not self.root(x):
        p = self.p[x]
        g = self.p[p]
        if not self.root(p):
            self.rotate(
                p if (self.ch[p][1] == x) == (self.ch[g][1] == p) else x
            )
        self.rotate(x)

def access(self, x):
    last = 0
    y = x
    while y:
        self.splay(y)
        self.ch[y][1] = last
        self.pull(y)
        last = y
        y = self.p[y]
    self.splay(x)

def makeroot(self, x):
    self.access(x)
    self.reverse(x)

def findroot(self, x):
    self.access(x)
    while self.push(x) is None and self.ch[x][0]:
        x = self.ch[x][0]
    self.splay(x)
    return x

def link(self, u, v):
    self.makeroot(u)
    self.p[u] = v

def cut(self, u, v):

```

```

        self.makeroot(u)
        self.access(v)
        self.ch[v][0] = 0
        self.p[u] = 0
        self.pull(v)

def path(self, u, v):
    self.makeroot(u)
    self.access(v)
    return v

def set(self, u, x):
    self.access(u)
    self.val[u] = x
    self.pull(u)

def main():
    d = sys.stdin.buffer.read().split()
    n, q = map(int, d[:2])
    t = LinkCut(list(map(int, d[2 : 2 + n])), "sum", None)
    at = 2 + n
    out = []
    for _ in range(q):
        op = d[at]
        at += 1
        if op == b"LINK":
            u, v = map(int, d[at : at + 2])
            at += 2
            t.link(u, v)
        elif op == b"CUT":
            u, v = map(int, d[at : at + 2])
            at += 2
            t.cut(u, v)
        elif op == b"SET":
            u, x = map(int, d[at : at + 2])
            at += 2
            t.set(u, x)
        elif op == b"AFFINE":
            u, v, a, b = map(int, d[at : at + 4])
            at += 4
            x = t.path(u, v)
            t.affine(x, a, b)
        else:
            u, v = map(int, d[at : at + 2])
            at += 2
            out.append(str(t.agg[t.path(u, v)]))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

E. Earliest Connection

How to find it

Label each Kruskal reconstruction merge by its input edge index.

Correctness

Two vertices first become connected at the merge represented by their reconstruction-tree LCA. If they have different final roots they never connect; identical vertices connect at time zero.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```
#include <bits/stdc++.h>
using namespace std;
struct KRT {
    int n, z, L;
    vector<int> dsu, parent, dep, weight, sz, component;
    vector<vector<int>> up, child;
    int find(int x) { return dsu[x] == x ? x : dsu[x] = find(dsu[x]); }
    KRT(int n, vector<array<int, 3>> e)
        : n(n), z(z), dsu(2 * n + 5), parent(2 * n + 5), dep(2 * n + 5),
          weight(2 * n + 5, -1), sz(2 * n + 5, 1), child(2 * n + 5) {
        iota(dsu.begin(), dsu.end(), 0);
        iota(parent.begin(), parent.end(), 0);
        sort(e.begin(), e.end());
        for (auto [w, u, v] : e) {
            u = find(u);
            v = find(v);
            if (u == v)
                continue;
            int x = z++;
            weight[x] = w;
            sz[x] = sz[u] + sz[v];
            child[x] = {u, v};
            parent[u] = parent[v] = x;
            dsu[u] = dsu[v] = dsu[x] = x;
        }
        vector<int> roots;
        for (int i = 0; i < z; i++)
            if (parent[i] == i)
                roots.push_back(i);
        for (int r : roots) {
            vector<int> st = {r};
            while (!st.empty()) {
                int x = st.back();
                st.pop_back();
                for (int y : child[x])
                    dep[y] = dep[x] + 1, st.push_back(y);
            }
        }
        L = max(1, (int)bit_width((unsigned)z));
        up.assign(L, vector<int>(z));
        for (int i = 0; i < z; i++)
            up[0][i] = parent[i];
        for (int j = 1; j < L; j++)
            for (int i = 0; i < z; i++)
                up[j][i] = up[j - 1][up[j - 1][i]];
        component.resize(n);
        for (int i = 0; i < n; i++)
            component[i] = find(i);
    }
    int climb(int v, int x) {
```

```

    for (int j = L - 1; j >= 0; j--) {
        int q = up[j][v];
        if (q != v && weight[q] <= x)
            v = q;
    }
    return v;
}

int lca(int a, int b) {
    if (component[a] != component[b])
        return -1;
    if (dep[a] < dep[b])
        swap(a, b);
    int d = dep[a] - dep[b];
    for (int j = 0; j < L; j++)
        if (d >> j & 1)
            a = up[j][a];
    if (a == b)
        return a;
    for (int j = L - 1; j >= 0; j--)
        if (up[j][a] != up[j][b])
            a = up[j][a], b = up[j][b];
    return up[0][a];
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<array<int, 3>> e(m);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        e[i] = {i + 1, u - 1, v - 1};
    }
    KRT t(n, e);
    while (q--) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        if (u == v)
            cout << 0 << '\n';
        else {
            int x = t.lca(u, v);
            cout << (x < 0 ? -1 : t.weight[x]) << '\n';
        }
    }
}

```

Python

```

import sys

class KRT:
    def __init__(self, n, edges):
        z = 2 * n + 5
        self.parent = list(range(z))
        self.tree = [[] for _ in range(z)]
        self.weight = [-1] * z

```

```

self.size = [1] * z
self.next = n
dsu = list(range(z))

def find(x):
    while dsu[x] != x:
        dsu[x] = dsu[dsu[x]]
        x = dsu[x]
    return x

for w, u, v in sorted(edges):
    a, b = find(u), find(v)
    if a == b:
        continue
    x = self.next
    self.next += 1
    self.weight[x] = w
    self.size[x] = self.size[a] + self.size[b]
    self.tree[x] = [a, b]
    self.parent[a] = self.parent[b] = x
    dsu[a] = dsu[b] = dsu[x] = x
roots = [v for v in range(self.next) if self.parent[v] == v]
self.depth = [0] * self.next
order = []
for root in roots:
    st = [root]
    while st:
        x = st.pop()
        order.append(x)
        for y in self.tree[x]:
            self.depth[y] = self.depth[x] + 1
            st.append(y)
self.L = max(1, self.next.bit_length())
self.up = [[0] * self.next for _ in range(self.L)]
self.up[0] = self.parent[: self.next]
for j in range(1, self.L):
    self.up[j] = [
        self.up[j - 1][self.up[j - 1][x]] for x in range(self.next)
    ]
self.component = [self.up[-1][v] for v in range(n)]

def climb(self, v, x):
    for j in range(self.L - 1, -1, -1):
        p = self.up[j][v]
        if p != v and self.weight[p] <= x:
            v = p
    return v

def lca(self, a, b):
    if self.component[a] != self.component[b]:
        return -1
    if self.depth[a] < self.depth[b]:
        a, b = b, a
    d = self.depth[a] - self.depth[b]
    for j in range(self.L):
        if d >> j & 1:
            a = self.up[j][a]
    if a == b:
        return a
    for j in range(self.L - 1, -1, -1):
        if self.up[j][a] != self.up[j][b]:
            a, b = self.up[j][a], self.up[j][b]
    return self.up[0][a]

```



```

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m, q = d[:3]
    at = 3
    e = []
    for w in range(1, m + 1):
        u, v = d[at : at + 2]
        at += 2
        e.append((w, u - 1, v - 1))
    t = KRT(n, e)
    out = []
    for i in range(at, at + 2 * q, 2):
        u, v = d[i] - 1, d[i + 1] - 1
        if u == v:
            out.append("0")
        else:
            x = t.lca(u, v)
            out.append(str(-1 if x < 0 else t.weight[x]))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

F. Affine Forest Paths

How to find it

Apply affine tags to an exposed path and compose a new tag after the pending old tag.

Correctness

For a subtree of size s , $x \rightarrow ax+b$ changes its sum to $a \cdot \text{sum} + b \cdot s$. Composition `new(old(x))` gives the stored multiplication and addition formulas, so delayed updates remain chronological.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \text{sigma})$ or $O(n)$ structure memory, as appropriate.

C++

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct LCT {
    int n, mode;
    ll mod;
    vector<array<int, 2>> ch;
    vector<int> p, rev, sz;
    vector<ll> val, agg, mul, add;
    LCT(vector<ll> a, int mode = 0, ll mod = 0)
        : n(a.size()), mode(mode), mod(mod), ch(n + 1), p(n + 1), rev(n + 1),
          sz(n + 1, 1), val(n + 1), agg(n + 1), mul(n + 1, 1), add(n + 1) {}
    sz[0] = 0;
    for (int i = 1; i <= n; i++)
        val[i] = agg[i] = a[i - 1];
}

```

```

bool root(int x) { return !p[x] || (ch[p[x]][0] != x && ch[p[x]][1] != x); }
void pull(int x) {
    auto [l, r] = ch[x];
    sz[x] = 1 + sz[l] + sz[r];
    agg[x] = mode ? (agg[l] ^ val[x] ^ agg[r]) : agg[l] + val[x] + agg[r];
    if (mod)
        agg[x] %= mod;
}
void reverse_node(int x) {
    if (x)
        swap(ch[x][0], ch[x][1]), rev[x] ^= 1;
}
void affine(int x, ll a, ll b) {
    if (!x)
        return;
    val[x] = (a * val[x] + b) % mod;
    agg[x] = (a * agg[x] + b * sz[x]) % mod;
    mul[x] = a * mul[x] % mod;
    add[x] = (a * add[x] + b) % mod;
}
void push(int x) {
    if (rev[x])
        reverse_node(ch[x][0]), reverse_node(ch[x][1]), rev[x] = 0;
    if (mul[x] != 1 || add[x]) {
        affine(ch[x][0], mul[x], add[x]);
        affine(ch[x][1], mul[x], add[x]);
        mul[x] = 1;
        add[x] = 0;
    }
}
void rotate(int x) {
    int y = p[x], z = p[y], s = ch[y][1] == x, b = ch[x][s ^ 1];
    if (!root(y))
        ch[z][ch[z][1] == y] = x;
    p[x] = z;
    ch[x][s ^ 1] = y;
    p[y] = x;
    ch[y][s] = b;
    if (b)
        p[b] = y;
    pull(y);
    pull(x);
}
void splay(int x) {
    vector<int> st = {x};
    for (int y = x; !root(y);)
        y = p[y], st.push_back(y);
    while (!st.empty())
        push(st.back(), st.pop_back());
    while (!root(x)) {
        int y = p[x], z = p[y];
        if (!root(y))
            rotate((ch[y][1] == x) == (ch[z][1] == y) ? y : x);
        rotate(x);
    }
}
void access(int x) {
    for (int y = 0, z = x; z; y = z, z = p[z])
        splay(z), ch[z][1] = y, pull(z);
    splay(x);
}
void makeroot(int x) {
    access(x);
}

```

```

    reverse_node(x);
}
void link(int x, int y) {
    makeroot(x);
    p[x] = y;
}
void cut(int x, int y) {
    makeroot(x);
    access(y);
    ch[y][0] = 0;
    p[x] = 0;
    pull(y);
}
int path(int x, int y) {
    makeroot(x);
    access(y);
    return y;
}
void setv(int x, ll v) {
    access(x);
    val[x] = v;
    pull(x);
}
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    LCT t(a, 0, 998244353);
    while (q--) {
        string op;
        cin >> op;
        if (op == "LINK") {
            int u, v;
            cin >> u >> v;
            t.link(u, v);
        } else if (op == "CUT") {
            int u, v;
            cin >> u >> v;
            t.cut(u, v);
        } else if (op == "SET") {
            int u;
            ll x;
            cin >> u >> x;
            t.setv(u, x);
        } else if (op == "AFFINE") {
            int u, v;
            ll x, y;
            cin >> u >> v >> x >> y;
            int z = t.path(u, v);
            t.affine(z, x, y);
        } else {
            int u, v;
            cin >> u >> v;
            cout << t.agg[t.path(u, v)] << '\n';
        }
    }
}
}

```

Python

```
import sys

class LinkCut:
    def __init__(self, values, mode="sum", mod=None):
        n = len(values)
        self.n = n
        self.ch = [[0, 0] for _ in range(n + 1)]
        self.p = [0] * (n + 1)
        self.rev = [0] * (n + 1)
        self.val = [0] + values[:]
        self.agg = [0] + values[:]
        self.sz = [0] + [1] * n
        self.mode = mode
        self.mod = mod
        self.mul = [1] * (n + 1)
        self.add = [0] * (n + 1)

    def root(self, x):
        p = self.p[x]
        return not p or (self.ch[p][0] != x and self.ch[p][1] != x)

    def pull(self, x):
        l, r = self.ch[x]
        self.sz[x] = 1 + self.sz[l] + self.sz[r]
        self.agg[x] = (
            (self.agg[l] ^ self.val[x] ^ self.agg[r])
            if self.mode == "xor"
            else self.agg[l] + self.val[x] + self.agg[r]
        )
        if self.mod:
            self.agg[x] %= self.mod

    def reverse(self, x):
        if x:
            self.ch[x].reverse()
            self.rev[x] ^= 1

    def affine(self, x, a, b):
        if not x:
            return
        mod = self.mod
        self.val[x] = (a * self.val[x] + b) % mod
        self.agg[x] = (a * self.agg[x] + b * self.sz[x]) % mod
        self.mul[x] = a * self.mul[x] % mod
        self.add[x] = (a * self.add[x] + b) % mod

    def push(self, x):
        if self.rev[x]:
            self.reverse(self.ch[x][0])
            self.reverse(self.ch[x][1])
            self.rev[x] = 0
        if self.mul[x] != 1 or self.add[x]:
            a, b = self.mul[x], self.add[x]
            self.affine(self.ch[x][0], a, b)
            self.affine(self.ch[x][1], a, b)
            self.mul[x] = 1
            self.add[x] = 0

    def rotate(self, x):
        p = self.p[x]
```

```

    g = self.p[p]
    side = self.ch[p][1] == x
    b = self.ch[x][side ^ 1]
    if not self.root(p):
        self.ch[g][self.ch[g][1] == p] = x
    self.p[x] = g
    self.ch[x][side ^ 1] = p
    self.p[p] = x
    self.ch[p][side] = b
    if b:
        self.p[b] = p
    self.pull(p)
    self.pull(x)

def splay(self, x):
    path = [x]
    y = x
    while not self.root(y):
        y = self.p[y]
        path.append(y)
    for y in reversed(path):
        self.push(y)
    while not self.root(x):
        p = self.p[x]
        g = self.p[p]
        if not self.root(p):
            self.rotate(
                p if (self.ch[p][1] == x) == (self.ch[g][1] == p) else x
            )
        self.rotate(x)

def access(self, x):
    last = 0
    y = x
    while y:
        self.splay(y)
        self.ch[y][1] = last
        self.pull(y)
        last = y
        y = self.p[y]
    self.splay(x)

def makeroot(self, x):
    self.access(x)
    self.reverse(x)

def findroot(self, x):
    self.access(x)
    while self.push(x) is None and self.ch[x][0]:
        x = self.ch[x][0]
    self.splay(x)
    return x

def link(self, u, v):
    self.makeroot(u)
    self.p[u] = v

def cut(self, u, v):
    self.makeroot(u)
    self.access(v)
    self.ch[v][0] = 0
    self.p[u] = 0
    self.pull(v)

```

```

def path(self, u, v):
    self.makeroot(u)
    self.access(v)
    return v

def set(self, u, x):
    self.access(u)
    self.val[u] = x
    self.pull(u)

def main():
    d = sys.stdin.buffer.read().split()
    n, q = map(int, d[:2])
    t = LinkCut(list(map(int, d[2 : 2 + n])), "sum", 998244353)
    at = 2 + n
    out = []
    for _ in range(q):
        op = d[at]
        at += 1
        if op == b"LINK":
            u, v = map(int, d[at : at + 2])
            at += 2
            t.link(u, v)
        elif op == b"CUT":
            u, v = map(int, d[at : at + 2])
            at += 2
            t.cut(u, v)
        elif op == b"SET":
            u, x = map(int, d[at : at + 2])
            at += 2
            t.set(u, x)
        elif op == b"AFFINE":
            u, v, a, b = map(int, d[at : at + 4])
            at += 4
            x = t.path(u, v)
            t.affine(x, a, b)
        else:
            u, v = map(int, d[at : at + 2])
            at += 2
            out.append(str(t.agg[t.path(u, v)]))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

Contest-wide validation

Compare A with sorted slices and prefix sums, B with direct element clamps, C with literal strings, D/F with BFS path reconstruction in an adjacency-list forest, and E with replayed edge prefixes. Include all-equal values, empty effective clamps, palindrome reversals, one-vertex paths, failed duplicate links, and many edges sharing one threshold.

The reusable habit is to separate represented state from storage state: wavelet partitions, treap rotations, splay preferred paths, and reconstruction nodes are correct only insofar as their summaries preserve the original query boundary.